SQL Interview Questions & Answers

For Data Analyst interviews (MySQL 8 syntax)

392 questions across 25 topics (plus 1 bonus), each with a clear answer or a worked SQL solution.
★ marks the questions asked most often in interviews.

How to use this guide

- Start with the starred questions and the **Must Know** list on the last page; do not try to memorise everything at once.
- For coding questions, read the question, try writing the query yourself on the sample schema, then compare with the solution. Understanding why a query works matters more than the exact syntax.
- Solutions are written for MySQL 8.0+ (window functions and CTEs need 8.0). Where other databases differ, a note says so.
- In interviews, state your assumptions first (how you define a metric, how ties are handled), then write the query step by step.

Sample schema used in the SQL answers

```
employees   (emp_id PK, emp_name, salary, dept_id FK, manager_id, gender, hire_date)
departments (dept_id PK, dept_name)
customers   (customer_id PK, customer_name, email, region, signup_date)
orders      (order_id PK, customer_id FK, order_date, amount, status)
order_items (order_id FK, product_id FK, quantity, unit_price)   -- composite key
products    (product_id PK, product_name, category, price, cost)
logins      (user_id, login_date)                                -- website activity
users       (user_id PK, signup_date)                            -- used in retention examples
```

Contents

1. SQL & Database Fundamentals 4
2. Keys and Constraints 5
3. Basic SQL Queries 6
4. NULL Values 8
5. Aggregate Functions 8
6. GROUP BY and HAVING 9
7. SQL Joins 10
8. Subqueries 12
9. CTEs - Common Table Expressions 13
10. CASE Statements 14
11. Window Functions 15
12. Date and Time Functions 17
13. String Functions 19
14. Data Cleaning Using SQL 20
15. UNION and Set Operations 22
16. Views 22
17. Indexes 23
18. Query Performance & Optimization 24
19. Transactions 25
20. Frequently Asked SQL Coding Questions 25
21. Intermediate SQL Problems 27

22. Advanced Analytical SQL Questions 29
23. Business/Data Analyst SQL Questions 32
24. SQL + Real-World Scenarios 36
25. MySQL-Specific Questions 38
Bonus: Extra Question from the "First 25" List 40
Quick Checklist: What to Master First 41

1. SQL & Database Fundamentals

1. ★ What is SQL?

SQL (Structured Query Language) is the standard language used to define, query, manipulate and control data stored in a relational database.

2. ★ What is a database?

An organized collection of structured data stored electronically so it can be easily accessed, managed and updated.

3. ★ What is DBMS?

A Database Management System is software that lets you create, store, retrieve, update and secure data in a database (e.g. MySQL, Oracle, MongoDB).

4. ★ What is RDBMS?

A Relational DBMS stores data in tables (rows and columns) and links tables through keys. It is queried with SQL. Examples: MySQL, PostgreSQL, SQL Server, Oracle.

5. ★ What is the difference between DBMS and RDBMS?

- DBMS: data may be stored as files, hierarchies or documents; few or no relationships between data; weaker integrity support.
- RDBMS: data in related tables; supports keys, constraints, normalization, ACID transactions and multi-user access.
- Every RDBMS is a DBMS, but not every DBMS is an RDBMS.

6. ★ What is MySQL?

An open-source Relational Database Management System (owned by Oracle) that uses SQL. It is widely used for web applications and analytics.

7. ★ What is the difference between SQL and MySQL?

SQL is the language. MySQL is a software product (an RDBMS) that understands and runs SQL. Other products (PostgreSQL, SQL Server) also use SQL, each with its own small dialect differences.

8. What is a relational database?

A database that organizes data into tables that are related to each other through primary and foreign keys.

9. What is a table?

The basic storage structure of a database: data arranged in rows and columns, describing one entity such as customers or orders.

10. What is a record/row?

A single entry in a table that holds one set of related values (e.g. one customer). Also called a tuple.

11. What is a column?

A vertical field in a table holding one attribute (e.g. email) with a defined data type. Also called a field or attribute.

12. What is a schema?

The logical blueprint of a database: its tables, columns, data types, relationships, views and constraints. In MySQL, "schema" and "database" mean the same thing.

13. What is a database server?

A machine or service that hosts databases, accepts client requests (queries), processes them and returns results, e.g. MySQL Server.

14. What are the different types of SQL commands?

DDL, DML, DQL, DCL and TCL (see the next question).

15. ★ What are DDL, DML, DQL, DCL, and TCL?

- DDL (Data Definition Language): defines structure. CREATE, ALTER, DROP, TRUNCATE, RENAME.
- DML (Data Manipulation Language): changes data. INSERT, UPDATE, DELETE.
- DQL (Data Query Language): retrieves data. SELECT.

- DCL (Data Control Language): permissions. GRANT, REVOKE.
- TCL (Transaction Control Language): manages transactions. COMMIT, ROLLBACK, SAVEPOINT.

16. What is the difference between structured and unstructured data?

Structured data fits a fixed schema of rows and columns (sales tables, customer lists). Unstructured data has no predefined format (emails, images, videos, free text, logs).

17. What is normalization?

The process of organizing tables to reduce redundancy and avoid insert/update/delete anomalies by splitting data into related tables.

18. What is denormalization?

Deliberately adding redundancy (combining tables or duplicating columns) to make reads and reporting faster, at the cost of extra storage and harder updates. Common in data warehouses.

19. What are the different normal forms?

- 1NF: atomic values, no repeating groups.
- 2NF: 1NF + no partial dependency on part of a composite key.
- 3NF: 2NF + no transitive dependency (non-key columns depend only on the key).
- BCNF: stricter 3NF; every determinant is a candidate key. 4NF and 5NF deal with multi-valued and join dependencies.

20. What is data integrity?

The accuracy, consistency and reliability of data over its lifecycle, enforced through constraints, keys, data types and transactions.

21. What is referential integrity?

A rule that a foreign key value must either match an existing primary key value in the parent table or be NULL. It prevents orphan records.

2. Keys and Constraints

22. ★ What is a primary key?

A column (or set of columns) that uniquely identifies each row. It cannot be NULL or duplicated, and a table has only one.

23. ★ What is a foreign key?

A column in one table that references the primary key (or a unique key) of another table, creating a relationship and enforcing referential integrity.

24. ★ What is the difference between PRIMARY KEY and UNIQUE KEY?

- A table can have only one primary key but many unique keys.
- Primary key cannot contain NULL; a unique key can (MySQL allows multiple NULLs).
- In InnoDB the primary key is the clustered index.

25. What is a composite key?

A key made of two or more columns that together uniquely identify a row, e.g. (order_id, product_id) in order_items.

26. What is a candidate key?

A minimal set of columns that can uniquely identify a row. A table can have several; one is chosen as the primary key.

27. What is a super key?

Any set of columns that uniquely identifies a row, even if it contains extra unnecessary columns. Every candidate key is a super key.

28. What is a surrogate key?

An artificial key with no business meaning, usually system generated (e.g. an AUTO_INCREMENT id).

29. What is a natural key?

A key built from real-world data that is naturally unique, such as email, passport number or ISBN.

30. ★ Can a primary key contain NULL values?

No. A primary key must be unique and NOT NULL.

31. Can a table have multiple primary keys?

No, only one primary key per table. But that one key can be composite (made of several columns).

32. Can a table have multiple UNIQUE keys?

Yes, a table can have many UNIQUE constraints.

33. What is a NOT NULL constraint?

It forces a column to always have a value; inserting NULL is rejected.

34. What is a DEFAULT constraint?

It supplies a default value for a column when none is given during INSERT, e.g. `status VARCHAR(10) DEFAULT 'new'`.

35. What is a CHECK constraint?

It restricts values to those satisfying a condition, e.g. `CHECK (salary > 0)`. MySQL enforces it from version 8.0.16.

36. What is a constraint?

A rule applied to a column or table to keep data valid: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT.

37. Why are foreign keys important?

They keep related tables consistent (no orders for non-existent customers), make relationships explicit, enable reliable joins, and can cascade updates/deletes.

3. Basic SQL Queries

38. ★ How do you retrieve all records from a table?

Use `SELECT * FROM table_name;`

39. ★ How do you select specific columns?

List the column names: `SELECT emp_name, salary FROM employees;`

40. ★ What is the difference between `SELECT *` and selecting specific columns?

`SELECT *` returns every column, which wastes memory and network, can break when the table changes, and prevents index-only scans. Selecting specific columns is faster, clearer and safer in production.

41. ★ How does the `WHERE` clause work?

It filters rows before grouping and aggregation, keeping only rows where the condition is true. Example: `SELECT * FROM employees WHERE salary > 50000;`

42. What is the difference between `WHERE` and `HAVING`?

`WHERE` filters individual rows before `GROUP BY`; `HAVING` filters groups after aggregation, so only `HAVING` can use aggregate functions.

43. What is the `DISTINCT` keyword?

It removes duplicate rows from the result: `SELECT DISTINCT dept_id FROM employees;`

44. ★ How do you find duplicate values?

Group by the column and keep groups that appear more than once.

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

45. How do you sort data?

With `ORDER BY column [ASC|DESC]`.

46. ★ What is `ORDER BY`?

A clause that sorts the result set by one or more columns. Default order is ascending. It is evaluated near the end of query processing.

47. What is the difference between ascending and descending order?

ASC sorts from smallest to largest (A to Z, oldest to newest) and is the default. DESC sorts largest to smallest (Z to A, newest to oldest).

48. What does `LIMIT` do?

It restricts the number of rows returned: `LIMIT 10` returns 10 rows; `LIMIT 10 OFFSET 20` skips 20 and returns the next 10.

49. How do you retrieve the top 5 records?

Sort and limit.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

50. How do you retrieve the second-highest value?

Skip the first distinct value with OFFSET (or use a subquery / window function).

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

To return NULL instead of an empty result when no second value exists, wrap it: `SELECT (SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1) AS second_highest;`

51. How do you retrieve the top 10 highest salaries?

`SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 10;` If you want the top 10 distinct salary values instead, use `SELECT DISTINCT salary ... LIMIT 10`.

52. How do you filter records using multiple conditions?

Combine conditions with AND / OR and use parentheses to control precedence.

```
SELECT * FROM employees
WHERE dept_id = 3 AND (salary > 60000 OR hire_date < '2020-01-01');
```

53. What are `AND`, `OR`, and `NOT`?

Logical operators. AND needs all conditions true, OR needs at least one true, NOT reverses a condition. AND has higher precedence than OR.

54. What is the `IN` operator?

It checks whether a value matches any value in a list or subquery: `WHERE dept_id IN (1, 2, 3)`. It is shorthand for several OR conditions.

55. What is the `BETWEEN` operator?

It tests a range, inclusive of both ends: `WHERE salary BETWEEN 40000 AND 60000`.

56. What is the `LIKE` operator?

It matches text patterns using wildcards: `WHERE emp_name LIKE 'A%'` finds names starting with A.

57. What are wildcards in SQL?

Special characters used with LIKE for pattern matching: `%` (any number of characters) and `_` (exactly one character).

58. What is the difference between `%` and `_` in `LIKE`?

`%` matches zero or more characters; `_` matches exactly one. `'A%'` matches Amit and Anjali; `'A_'` matches only two-letter names starting with A.

4. NULL Values

59. ★ What is NULL in SQL?

NULL represents a missing, unknown or not-applicable value. It is not a value; it is the absence of one.

60. ★ What is the difference between NULL and 0?

0 is a known numeric value; NULL means unknown. `SUM`/`AVG` count 0 but ignore NULL, and `NULL = 0` is not true.

61. ★ What is the difference between NULL and an empty string?

An empty string `''` is a known value of length 0; NULL means no value. `LENGTH('')` is 0 while `LENGTH(NULL)` is NULL, and `COUNT(column)` counts empty strings but not NULLs.

62. How do you find NULL values?

```
SELECT * FROM customers WHERE email IS NULL;
```

63. How do you find non-NULL values?

```
SELECT * FROM customers WHERE email IS NOT NULL;
```

64. ★ What is `IS NULL`?

The only correct way to test for NULL. `= NULL` never works because any comparison with NULL returns UNKNOWN.

65. ★ What is `IS NOT NULL`?

It returns rows where the column has a value (is not NULL).

66. What is `COALESCE()`?

It returns the first non-NULL argument: `COALESCE(phone, mobile, 'N/A')`. It is standard SQL and accepts many arguments.

67. What is `IFNULL()` in MySQL?

A MySQL function with two arguments: `IFNULL(expr, replacement)` returns replacement when expr is NULL.

68. What is the difference between `COALESCE()` and `IFNULL()`?

`COALESCE` is standard SQL, takes any number of arguments and returns the first non-NULL. `IFNULL` is MySQL-specific and takes exactly two.

69. How do NULL values behave in calculations?

Any arithmetic with NULL gives NULL (`5 + NULL` is NULL). Comparisons with NULL give UNKNOWN, so those rows are filtered out of WHERE.

70. How does NULL affect aggregate functions?

SUM, AVG, MIN, MAX and COUNT(column) ignore NULLs. COUNT(*) counts all rows. AVG divides by the count of non-NULL values only, which can change your result.

71. How would you replace NULL values?

In a query, use `COALESCE(col, 0)` or `IFNULL(col, 0)`. To change stored data: `UPDATE t SET col = 0 WHERE col IS NULL;`

5. Aggregate Functions

72. ★ What are aggregate functions?

Functions that calculate a single result from a set of rows: COUNT, SUM, AVG, MIN, MAX (and GROUP_CONCAT in MySQL).

73. ★ What is `COUNT()`?

It counts rows or non-NULL values.

74. ★ What is the difference between `COUNT(*)` and `COUNT(column)`?

`COUNT(*)` counts all rows including those with NULLs. `COUNT(column)` counts only rows where that column is not NULL.

75. What is `COUNT(DISTINCT column)`?

It counts the unique non-NULL values: `COUNT(DISTINCT customer_id)` gives the number of unique customers.

76. ★ What is `SUM()`?

It adds up numeric values in a column, ignoring NULLs.

77. ★ What is `AVG()`?

It returns the average of non-NULL numeric values (sum divided by count of non-NULL values).

78. ★ What is `MIN()`?

It returns the smallest value in a column (also works on dates and text).

79. ★ What is `MAX()`?

It returns the largest value in a column (also works on dates and text).

80. How do aggregate functions handle NULL values?

They skip NULLs (except COUNT(*)). If every value is NULL, SUM/AVG/MIN/MAX return NULL and COUNT(column) returns 0.

81. How do you calculate the average salary?

```
SELECT AVG(salary) FROM employees;
```

82. How do you find the maximum salary?

```
SELECT MAX(salary) FROM employees;
```

83. How do you find the minimum salary?

```
SELECT MIN(salary) FROM employees;
```

84. How do you count unique customers?

```
SELECT COUNT(DISTINCT customer_id) FROM orders;
```

85. How do you calculate total sales?

```
SELECT SUM(amount) AS total_sales FROM orders;
```

6. GROUP BY and HAVING

86. ★ What is `GROUP BY`?

A clause that groups rows sharing the same values in the given columns so aggregate functions can be applied to each group.

87. ★ Why do we use `GROUP BY`?

To summarise data per category: sales per region, employees per department, orders per month.

88. ★ What is the difference between `WHERE` and `HAVING`?

- WHERE filters rows before grouping; cannot use aggregates.
- HAVING filters groups after aggregation; can use aggregates.
- Order of execution: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.

89. Can we use `WHERE` with aggregate functions?

No. WHERE runs before aggregation. Use HAVING, or a subquery/CTE.

90. Can we use `HAVING` without `GROUP BY`?

Yes. The whole table is treated as one group, e.g. `SELECT COUNT(*) FROM employees HAVING COUNT(*) > 10;`. It is rare in practice.

91. ★ How do you find the number of employees in each department?

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id;
```

92. How do you find departments having more than 10 employees?

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 10;
```

93. How do you find the average salary by department?

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
```

94. How do you find the highest salary in each department?

```
SELECT dept_id, MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id;
```

95. How do you find customers who placed more than 5 orders?

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

96. How do you group data by multiple columns?

List several columns in GROUP BY; each unique combination forms a group.

```
SELECT region, YEAR(order_date) AS yr, SUM(amount) AS revenue
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
GROUP BY region, YEAR(order_date);
```

7. SQL Joins

Joins are among the most tested topics. Practice drawing the Venn diagram for each one.

97. ★ What is a JOIN?

A JOIN combines rows from two or more tables based on a related column (usually primary key = foreign key).

98. ★ Why are joins used?

Because normalized data is spread across tables. Joins bring it back together, e.g. orders with customer names.

99. ★ What is an INNER JOIN?

Returns only rows that have a match in both tables.

```
SELECT c.customer_name, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

100. ★ What is a LEFT JOIN?

Returns all rows from the left table and matching rows from the right; unmatched right-side columns are NULL.

101. ★ What is a RIGHT JOIN?

Returns all rows from the right table and matching rows from the left; unmatched left-side columns are NULL. It is the mirror of LEFT JOIN, so most people just swap table order and use LEFT JOIN.

102. What is a FULL OUTER JOIN?

Returns all rows from both tables, with NULLs where there is no match. MySQL has no FULL OUTER JOIN; emulate it with `LEFT JOIN ... UNION ... RIGHT JOIN ...`.

103. ★ What is a CROSS JOIN?

Returns the Cartesian product: every row of table A paired with every row of table B (m x n rows). Useful for generating combinations, e.g. all dates x all products.

104. What is a SELF JOIN?

Joining a table to itself using aliases, e.g. employees to their managers.

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

105. ★ Difference between INNER JOIN and LEFT JOIN?

INNER JOIN keeps only matching rows. LEFT JOIN keeps every left-table row and fills NULLs when there is no match.

106. ★ Difference between LEFT JOIN and RIGHT JOIN?

They differ only in which table is preserved: LEFT keeps all rows of the left table, RIGHT keeps all rows of the right table.

107. When would you use a LEFT JOIN?

When you must keep all rows from the main table even without matches: all customers including those with no orders, or to find unmatched records with `WHERE right.key IS NULL`.

108. How do you join three tables?

Chain the joins.

```
SELECT c.customer_name, o.order_id, p.product_name
FROM customers c
JOIN orders o       ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p     ON oi.product_id = p.product_id;
```

109. Can you join a table with itself?

Yes, with a self join using two different aliases for the same table.

110. What happens when there is no matching record in a JOIN?

INNER JOIN drops the row. LEFT/RIGHT/FULL OUTER JOIN keep it and show NULL for the missing side's columns.

111. ★ How do you find records that exist in one table but not another?

Use an anti-join: LEFT JOIN with IS NULL, NOT EXISTS, or NOT IN (careful with NULLs in NOT IN).

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

112. How do you find customers who have never placed an order?

Same anti-join as above, or `SELECT * FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);`

113. How do you find employees who don't belong to any department?

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

If the column is simply NULL, use `WHERE dept_id IS NULL`.

114. How do you identify unmatched records?

LEFT JOIN and filter `right_table.key IS NULL` (records only in the left); swap tables for the other direction; or use FULL OUTER JOIN emulation to see both sides.

115. What is a many-to-many relationship?

Many rows in A relate to many rows in B (students and courses, orders and products). Implemented with a junction/bridge table holding both foreign keys.

116. What is a one-to-many relationship?

One row in the parent table relates to many rows in the child table (one customer, many orders). The foreign key sits on the "many" side.

117. What is a one-to-one relationship?

Each row in A relates to at most one row in B (user and user_profile). Usually the foreign key is also unique.

118. How can joins create duplicate rows?

When the join key is not unique on one side (one-to-many or many-to-many), each match produces a row, so the "one" side's values repeat. Aggregating afterwards (e.g. SUM) then double counts.

119. How do you avoid duplicate rows after a JOIN?

- Join on the correct, unique keys.
- Aggregate the many side first in a CTE/subquery, then join.
- Use `DISTINCT` or `COUNT(DISTINCT ...)` when appropriate.
- Check row counts before and after the join.

8. Subqueries

120. ★ What is a subquery?

A query nested inside another query (in SELECT, FROM, WHERE or HAVING). The inner query runs and its result is used by the outer query.

121. ★ What is the difference between a subquery and a JOIN?

A JOIN combines columns from multiple tables into one result. A subquery returns a value, list or table to be used by an outer query. Joins are often faster and more readable for combining data; subqueries are handy for filtering or step-wise logic. Modern optimizers often treat them similarly.

122. What is a correlated subquery?

A subquery that references columns of the outer query, so it runs once per outer row.

```
SELECT e.emp_name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

123. What is a non-correlated subquery?

An independent subquery that runs once and does not use outer-query columns.

124. Where can you use a subquery?

In SELECT (scalar), FROM (derived table), WHERE/HAVING (with IN, EXISTS, =, >, ANY, ALL), and in INSERT/UPDATE/DELETE.

125. Can a subquery be used in `SELECT`?

Yes, as a scalar subquery returning exactly one value: `SELECT emp_name, (SELECT MAX(salary) FROM employees) AS top_salary FROM employees;`

126. Can a subquery be used in `FROM`?

Yes, as a derived table, which must have an alias: `SELECT * FROM (SELECT dept_id, AVG(salary) a FROM employees GROUP BY dept_id) t;`

127. Can a subquery be used in `WHERE`?

Yes: `WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name = 'Sales')`.

128. ★ What is the difference between `IN` and `EXISTS`?

- IN compares a value to a list produced by a subquery.
- EXISTS checks whether the subquery returns any row; it stops at the first match and ignores the selected values.
- NOT IN returns nothing if the list contains a NULL; NOT EXISTS handles NULLs safely.
- EXISTS is usually better for large subquery results; IN is fine for small lists.

129. ★ When would you use `EXISTS`?

For existence checks and anti-joins, especially with correlated subqueries and large tables, e.g. customers who have at least one order, or none (NOT EXISTS).

130. How do you find employees earning more than the average salary?

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

131. How do you find the second-highest salary using a subquery?

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

132. How do you find customers whose spending is above the average?

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > (
    SELECT AVG(total) FROM (
        SELECT SUM(amount) AS total FROM orders GROUP BY customer_id
    ) t
);
```

9. CTEs - Common Table Expressions

133. ★ What is a CTE?

A named temporary result set defined with `WITH` that exists only for the duration of one query. Supported in MySQL 8.0+.

134. ★ Why do we use CTEs?

To make complex queries readable, break logic into steps, reuse a result several times in the same query, and write recursive queries.

135. What is the syntax of a CTE?

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
)
SELECT * FROM dept_avg WHERE avg_salary > 50000;
```

136. What is the difference between a CTE and a subquery?

A CTE is named, defined at the top, can be referenced multiple times and can be recursive; a subquery is inline, used where it is written, and can't be reused. They usually perform the same; CTEs win on readability.

137. What are the advantages of CTEs?

Readability, modular logic, reuse within a query, easier debugging, and recursion support.

138. Can we use multiple CTEs?

Yes, separate them with commas after a single `WITH`.

139. Can a CTE reference another CTE?

Yes, a CTE can use any CTE defined before it in the same WITH clause.

```
WITH a AS (SELECT ...),
     b AS (SELECT * FROM a WHERE ...)
SELECT * FROM b;
```

140. What is a recursive CTE?

A CTE that references itself: an anchor member plus a recursive member joined with UNION ALL, repeating until no new rows are produced. Declared with `WITH RECURSIVE` in MySQL.

141. When would you use a recursive CTE?

For hierarchical or sequential data: org charts (employee-manager chains), category trees, bill of materials, generating date series.

```
WITH RECURSIVE org AS (
    SELECT emp_id, emp_name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.emp_name, e.manager_id, o.level + 1
    FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org;
```

142. How can CTEs make SQL queries easier to understand?

Each CTE gets a meaningful name and one job (clean, aggregate, rank), so the query reads top to bottom like steps instead of deeply nested subqueries.

143. How would you use a CTE to calculate the top 3 customers?

```
WITH spend AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders
    GROUP BY customer_id
)
SELECT customer_id, total_spent
FROM spend
ORDER BY total_spent DESC
LIMIT 3;
```

10. CASE Statements

144. ★ What is a `CASE` statement?

SQL's if-then-else logic. It evaluates conditions in order and returns the result of the first true one, otherwise the ELSE value (or NULL if there is no ELSE).

145. ★ Why is `CASE` used in SQL?

To create categories/buckets, conditional columns, conditional aggregation (pivot-style counts), custom sort orders and NULL handling inside a single query.

146. What is the difference between simple CASE and searched CASE?

- Simple CASE compares one expression to values: `CASE grade WHEN 'A' THEN 'Top' WHEN 'B' THEN 'Good' END`.
- Searched CASE evaluates independent boolean conditions: `CASE WHEN score >= 90 THEN 'A' WHEN score >= 80 THEN 'B' END`. It is more flexible (ranges, multiple columns).

147. How do you categorize customers based on spending?

```
SELECT customer_id, SUM(amount) AS total_spent,
       CASE WHEN SUM(amount) >= 10000 THEN 'Gold'
            WHEN SUM(amount) >= 5000  THEN 'Silver'
            ELSE 'Bronze' END AS tier
FROM orders
GROUP BY customer_id;
```

148. How do you create an age group using CASE?

```
SELECT name, age,
       CASE WHEN age < 18 THEN 'Under 18'
            WHEN age BETWEEN 18 AND 34 THEN '18-34'
            WHEN age BETWEEN 35 AND 54 THEN '35-54'
            ELSE '55+' END AS age_group
FROM users;
```

149. How do you classify employees as High/Medium/Low salary?

```
SELECT emp_name, salary,
       CASE WHEN salary >= 80000 THEN 'High'
            WHEN salary >= 40000 THEN 'Medium'
            ELSE 'Low' END AS salary_band
FROM employees;
```

150. How do you count records conditionally using CASE?

Put CASE inside SUM or COUNT.

```
SELECT SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
       COUNT(CASE WHEN status = 'Completed' THEN 1 END)        AS completed_orders
FROM orders;
```

151. How do you calculate conditional percentages?

```
SELECT ROUND(100 * SUM(CASE WHEN status = 'Cancelled' THEN 1 ELSE 0 END) / COUNT(*), 2)
       AS cancellation_pct
FROM orders;
```

152. How can CASE be used with aggregate functions?

Inside the aggregate (conditional aggregation) to compute several filtered metrics in one pass, or to pivot rows into columns: `SUM(CASE WHEN region='North' THEN amount END) AS north_sales`.

11. Window Functions

Window functions calculate across a set of related rows WITHOUT collapsing them into one row.

153. ★ What is a window function?

A function that performs a calculation over a "window" of rows related to the current row, defined by `OVER (PARTITION BY ... ORDER BY ...)`, while still returning every row. Available in MySQL 8.0+.

154. ★ How is a window function different from GROUP BY?

GROUP BY collapses rows into one row per group. A window function keeps every row and adds the calculated value alongside, so you can show row detail and group total together.

155. ★ What is `ROW_NUMBER()`?

Assigns a unique sequential number (1, 2, 3...) to rows within a partition; ties get different numbers (arbitrarily unless ordering is unique).

156. ★ What is `RANK()`?

Assigns ranks with ties getting the same rank and leaving gaps afterwards (1, 2, 2, 4).

157. ★ What is `DENSE_RANK()`?

Like RANK but with no gaps after ties (1, 2, 2, 3).

158. ★ Difference between `RANK()` and `DENSE_RANK()`?

Both give ties the same rank. RANK skips the next numbers (1,2,2,4); DENSE_RANK does not (1,2,2,3). Use DENSE_RANK for "Nth highest" questions.

159. ★ Difference between `ROW_NUMBER()` and `RANK()`?

ROW_NUMBER never repeats a number even for ties; RANK gives tied rows the same number and skips following ranks.

160. What is `PARTITION BY`?

It splits the rows into independent groups (windows); the function restarts for each partition, e.g. rank within each department.

161. What is `ORDER BY` inside a window function?

It defines the order of rows inside each partition, which matters for ranking, LAG/LEAD, running totals and moving averages. It does not sort the final result unless you add an outer ORDER BY.

162. What is `LEAD()`?

Returns a value from a following row in the partition: `LEAD(amount, 1) OVER (ORDER BY order_date)` gives the next row's amount.

163. What is `LAG()`?

Returns a value from a preceding row: `LAG(amount, 1) OVER (ORDER BY order_date)` gives the previous row's amount. A third argument sets the default when no previous row exists.

164. What is `FIRST_VALUE()`?

Returns the first value in the window frame according to the ordering, e.g. each customer's first order amount.

165. What is `LAST_VALUE()`?

Returns the last value in the window frame. Gotcha: the default frame ends at the current row, so specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to get the true last value of the partition.

166. What is a running total?

A cumulative sum that adds each row's value to the sum of all earlier rows (e.g. year-to-date sales).

167. How do you calculate a running total?

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
```

168. How do you calculate a moving average?

Use a frame; for a 7-row moving average:

```
SELECT order_date, amount,
       AVG(amount) OVER (ORDER BY order_date
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7
FROM daily_sales;
```

169. How do you find the top 3 employees in each department?

```
SELECT * FROM (
    SELECT emp_name, dept_id, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk <= 3;
```

170. ★ How do you find the second-highest salary in each department?

```
SELECT dept_id, emp_name, salary
FROM (
    SELECT dept_id, emp_name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;
```

171. How do you compare current sales with previous month's sales?

Aggregate to month first, then use LAG.

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS sales
    FROM orders GROUP BY ym
)
SELECT ym, sales, LAG(sales) OVER (ORDER BY ym) AS prev_month_sales
FROM monthly;
```

172. How do you calculate month-over-month growth?

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS sales
    FROM orders GROUP BY ym
)
SELECT ym, sales,
       ROUND(100 * (sales - LAG(sales) OVER (ORDER BY ym))
             / LAG(sales) OVER (ORDER BY ym), 2) AS mom_growth_pct
FROM monthly;
```

173. How do you calculate cumulative sales?

Use `SUM(amount) OVER (ORDER BY order_date)` on daily or monthly totals (see running total).

174. How do you identify the first order of each customer?

```
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date, order_id) AS rn
    FROM orders o
) t
WHERE rn = 1;
```

175. How do you identify the latest order of each customer?

Same pattern with descending order: `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC, order_id DESC)` and keep `rn = 1`.

12. Date and Time Functions

176. ★ How do you work with dates in SQL?

Store them in DATE/DATETIME/TIMESTAMP columns and use date functions to extract parts (YEAR, MONTH), do arithmetic (DATE_ADD, DATEDIFF), format (DATE_FORMAT) and parse (STR_TO_DATE). Compare using ranges so indexes can be used.

177. ★ What is `DATE()`?

Extracts the date part from a datetime: `DATE('2025-03-15 14:30:00')` returns `2025-03-15`.

178. What is `YEAR()`?

Returns the year of a date: `YEAR('2025-03-15')` is 2025.

179. What is `MONTH()`?

Returns the month number 1-12: `MONTH('2025-03-15')` is 3.

180. What is `DAY()`?

Returns the day of the month (1-31). `DAYOFMONTH()` is a synonym.

181. What is `MONTHNAME()`?

Returns the month's name: `MONTHNAME('2025-03-15')` is March.

182. What is `DAYNAME()`?

Returns the weekday name: `DAYNAME('2025-03-15')` is Saturday.

183. What is `DATEDIFF()`?

Returns the number of days between two dates: `DATEDIFF(date1, date2)` is date1 minus date2.

184. What is `DATE_ADD()`?

Adds an interval to a date: `DATE_ADD('2025-01-31', INTERVAL 1 MONTH)`.

185. What is `DATE_SUB()`?

Subtracts an interval: `DATE_SUB(CURDATE(), INTERVAL 30 DAY)`.

186. What is `TIMESTAMPDIFF()`?

Returns the difference between two datetimes in a chosen unit: `TIMESTAMPDIFF(MONTH, start_date, end_date)`. The unit can be SECOND, MINUTE, HOUR, DAY, MONTH, YEAR, etc.

187. How do you extract the year from a date?

`SELECT YEAR(order_date) FROM orders;` (or `EXTRACT(YEAR FROM order_date)`).

188. How do you extract the month?

`SELECT MONTH(order_date) FROM orders;` or `MONTHNAME(order_date)` for the name.

189. How do you calculate the difference between two dates?

`DATEDIFF(end_date, start_date)` for days, `TIMESTAMPDIFF(unit, start, end)` for other units.

190. How do you find records from the last 30 days?

```
SELECT * FROM orders
WHERE order_date >= CURDATE() - INTERVAL 30 DAY;
```

191. How do you find customers who purchased this month?

Use a range so an index on order_date can be used.

```
SELECT DISTINCT customer_id
FROM orders
WHERE order_date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
  AND order_date <  DATE_FORMAT(CURDATE(), '%Y-%m-01') + INTERVAL 1 MONTH;
```

192. How do you calculate monthly sales?

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(amount) AS monthly_sales
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

193. How do you calculate yearly sales?

`SELECT YEAR(order_date) AS yr, SUM(amount) AS yearly_sales FROM orders GROUP BY YEAR(order_date);`

194. How do you find the busiest month?

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, COUNT(*) AS orders_cnt
FROM orders
GROUP BY month
ORDER BY orders_cnt DESC
LIMIT 1;
```

Use SUM(amount) instead of COUNT(*) if busiest means highest revenue.

195. How do you calculate customer retention by month?

Count customers active in a month who are also active the next month.

```
WITH m AS (
    SELECT DISTINCT customer_id,
           DATE_SUB(order_date, INTERVAL DAY(order_date) - 1 DAY) AS mth
    FROM orders
)
SELECT a.mth,
       COUNT(DISTINCT a.customer_id) AS active_customers,
       COUNT(DISTINCT b.customer_id) AS retained_next_month,
       ROUND(100 * COUNT(DISTINCT b.customer_id) / COUNT(DISTINCT a.customer_id), 2) AS retention_pct
FROM m a
LEFT JOIN m b ON a.customer_id = b.customer_id
             AND b.mth = DATE_ADD(a.mth, INTERVAL 1 MONTH)
GROUP BY a.mth
ORDER BY a.mth;
```

196. How do you calculate the average number of days between orders?

```
WITH gaps AS (
    SELECT customer_id,
           DATEDIFF(order_date,
                    LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)) AS days_gap
    FROM orders
)
SELECT AVG(days_gap) AS avg_days_between_orders
FROM gaps
WHERE days_gap IS NOT NULL;
```

13. String Functions

197. ★ What are string functions?

Built-in functions that manipulate text: change case, trim, join, slice, search and replace (CONCAT, LOWER, UPPER, TRIM, SUBSTRING, REPLACE, LENGTH...).

198. What is `CONCAT()`?

Joins strings together: `CONCAT(first_name, ' ', last_name)`. Returns NULL if any argument is NULL.

199. What is `CONCAT_WS()`?

Concatenate With Separator: `CONCAT_WS('-', a, b, c)`. It skips NULL values (but not empty strings).

200. What is `LENGTH()`?

Returns the length in bytes. For character count use `CHAR_LENGTH()`. They differ for multi-byte characters.

201. What is `LOWER()`?

Converts text to lowercase: `LOWER('SQL')` returns sql.

202. What is `UPPER()`?

Converts text to uppercase.

203. What is `TRIM()`?

Removes leading and trailing spaces (or a specified character): `TRIM(' hi ')` returns hi. `TRIM(BOTH 'x' FROM 'xxhixx')` returns hi.

204. What is `LTRIM()`?

Removes leading (left) spaces.

205. What is `RTRIM()`?

Removes trailing (right) spaces.

206. What is `SUBSTRING()`?

Extracts part of a string: `SUBSTRING('Analytics', 1, 4)` returns Anal. Positions start at 1.

207. What is `LEFT()`?

Returns the first N characters: `LEFT('Analytics', 3)` returns Ana.

208. What is `RIGHT()`?

Returns the last N characters: `RIGHT('Analytics', 3)` returns ics.

209. What is `REPLACE()`?

Replaces all occurrences of a substring: `REPLACE('a-b-c', '-', '')` returns abc.

210. What is `CHAR_LENGTH()`?

Returns the number of characters (not bytes) in a string.

211. How do you remove leading/trailing spaces?

`SELECT TRIM(col) FROM t;` or fix stored data with `UPDATE t SET col = TRIM(col);`

212. How do you extract part of a string?

Use SUBSTRING, LEFT/RIGHT or SUBSTRING_INDEX, e.g. the domain from an email: `SUBSTRING_INDEX(email, '@', -1)`.

213. How do you combine first name and last name?

`SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;`

214. How do you replace a particular character?

`SELECT REPLACE(phone, '-', '') FROM customers;`

215. How do you identify duplicate names with different capitalization?

Normalize case and spacing before grouping.

```
SELECT LOWER(TRIM(customer_name)) AS clean_name, COUNT(*) AS cnt
FROM customers
GROUP BY LOWER(TRIM(customer_name))
HAVING COUNT(*) > 1;
```

14. Data Cleaning Using SQL

Data cleaning questions are very practical: interviewers want to see how you handle messy real-world data.

216. ★ How do you identify duplicate records?

Group by the columns that define a duplicate and keep groups with COUNT(*) > 1; or use ROW_NUMBER() and look at rn > 1.

```
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;
```

217. ★ How do you remove duplicate records?

Keep one row per duplicate group (e.g. the lowest id) and delete the rest. Always preview with a SELECT and work inside a transaction/backup first.

```
DELETE c1
FROM customers c1
JOIN customers c2
  ON c1.email = c2.email
 AND c1.customer_id > c2.customer_id;
```

218. ★ How do you handle NULL values?

Depends on context: find them with IS NULL, replace with COALESCE/IFNULL (default or average), exclude them with IS NOT NULL, or leave them if NULL is meaningful. Document the choice.

219. How do you find inconsistent values?

Profile categorical columns with `SELECT col, COUNT(*) FROM t GROUP BY col ORDER BY col;` and look for variants such as "USA", "U.S.A", "usa ".

220. How do you remove leading/trailing spaces?

`UPDATE t SET col = TRIM(col);`

221. How do you standardize text values?

Combine TRIM, LOWER/UPPER, REPLACE and CASE to map variants to one value.

```
UPDATE customers
SET country = CASE WHEN LOWER(TRIM(country)) IN ('usa', 'u.s.a', 'united states') THEN 'USA'
                   ELSE TRIM(country) END;
```

222. How do you convert text to lowercase?

`LOWER(col)`, or `UPDATE t SET col = LOWER(col);`

223. How do you convert text to uppercase?

`UPPER(col)`, or `UPDATE t SET col = UPPER(col);`

224. How do you replace incorrect values?

Use UPDATE with CASE or REPLACE, e.g. `UPDATE t SET city = 'Bengaluru' WHERE city IN ('Bangalore', 'Bengalore');`

225. How do you identify invalid dates?

If dates are stored as text, parse them; STR_TO_DATE returns NULL for invalid ones.

```sql
SELECT raw_date
FROM staging
WHERE raw_date IS NOT NULL
  AND STR_TO_DATE(raw_date, '%d-%m-%Y') IS NULL;
```

226. How do you find outliers using SQL?

Use a z-score (|value - mean| / stddev > 3) or the IQR rule. Z-score example:

```sql
SELECT * FROM (
    SELECT order_id, amount,
           (amount - AVG(amount) OVER ()) / STDDEV(amount) OVER () AS z
    FROM orders
) t
WHERE ABS(z) > 3;
```

227. How do you check the uniqueness of a column?

`SELECT COUNT(*) = COUNT(DISTINCT col) AS is_unique FROM t;` or look for duplicates with GROUP BY HAVING.

228. How do you identify missing values?

`SELECT SUM(col IS NULL) AS null_count FROM t;` Also check for empty strings and placeholders like 'N/A' or 'unknown'.

229. How do you split a column into multiple columns?

```sql
SELECT SUBSTRING_INDEX(full_name, ' ', 1)  AS first_name,
       SUBSTRING_INDEX(full_name, ' ', -1) AS last_name
FROM customers;
```

230. How do you combine multiple columns?

`CONCAT(col1, ' ', col2)` or `CONCAT_WS(' ', col1, col2)`.

231. How do you convert a string into a date?

`STR_TO_DATE('15-08-2025', '%d-%m-%Y')` returns 2025-08-15.

232. How do you convert a string into a number?

`CAST(price_text AS DECIMAL(10,2))`, `CAST(qty AS UNSIGNED)` or `price_text + 0`. Clean the text first with REPLACE(col, ',', '').

233. What is `CAST()`?

Standard SQL function converting a value to another data type: `CAST(value AS type)`.

234. What is `CONVERT()`?

MySQL function for type conversion `CONVERT(value, type)` and character-set conversion `CONVERT(text USING utf8mb4)`. CAST is more portable.

235. How would you clean a messy dataset using SQL?

- Profile it first: row counts, NULLs, distinct values, min/max.
- Work on a copy or staging table, never the raw data.
- Trim and standardize text; fix data types and date formats.
- Handle NULLs and placeholders; fix invalid values.
- Remove duplicates with ROW_NUMBER or a self join.
- Check outliers; validate with counts and constraints; document each step.

15. UNION and Set Operations

236. ★ What is `UNION`?

Combines the results of two or more SELECTs into one result set and removes duplicate rows.

237. ★ What is `UNION ALL`?

Combines results and keeps all rows including duplicates. It is faster because it skips duplicate removal.

238. ★ Difference between `UNION` and `UNION ALL`?

UNION removes duplicates (extra sort/compare work); UNION ALL keeps them and is faster. Use UNION ALL unless you specifically need de-duplication.

239. What is `INTERSECT`?

Returns only rows that appear in both queries' results. Supported in MySQL 8.0.31+. Alternative: INNER JOIN or `WHERE ... IN`.

240. What is `EXCEPT`?

Returns rows from the first query that are not in the second (called MINUS in Oracle). Supported in MySQL 8.0.31+. Alternative: LEFT JOIN ... IS NULL or NOT EXISTS.

241. When would you use UNION?

When combining similar datasets and you need unique rows, e.g. a list of unique emails from two sources.

242. When would you use UNION ALL?

When duplicates are impossible or wanted, e.g. stacking monthly tables, or combining sales from different regions for totals.

243. What conditions must be satisfied when using UNION?

Each SELECT must have the same number of columns, with compatible data types in the same order. Column names come from the first SELECT.

244. Does UNION remove duplicates?

Yes.

245. Does UNION ALL remove duplicates?

No.

16. Views

246. ★ What is a View?

A saved SELECT query stored as a virtual table. It stores no data itself; it runs the query each time it is used.

247. Why do we use Views?

To simplify complex queries, reuse logic, hide sensitive columns, give consistent business definitions and provide a stable interface to changing tables.

248. What is the difference between a table and a View?

A table physically stores data; a view is a stored query that shows data from tables when queried.

249. What is a materialized view?

A view whose result is physically stored and refreshed periodically, giving much faster reads. PostgreSQL and Oracle support them natively; MySQL does not (you can emulate with a summary table).

250. Can we update a View?

Sometimes. Simple views (single table, no aggregates, DISTINCT, GROUP BY or window functions) are updatable; complex views are read-only.

251. What are the advantages of Views?

Simplicity, reusability, security (column/row restriction), consistency of definitions and abstraction from the underlying schema.

252. What are the disadvantages of Views?

Can hide slow queries, nested views hurt performance, limited updatability, and no stored data (so no speedup) in MySQL.

253. When would you create a View for a Data Analyst project?

For a reusable, clean dataset: e.g. `vw_monthly_sales` for a dashboard, or a cleaned customers view that handles trimming, casing and joins so every report uses the same logic.

```
CREATE VIEW vw_monthly_sales AS
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(amount) AS revenue
FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

17. Indexes

254. ★ What is an index?

A data structure (usually a B-tree) on one or more columns that lets the database find rows quickly without scanning the whole table, like a book's index.

255. ★ Why are indexes used?

To speed up SELECT queries, joins, WHERE filters, ORDER BY and GROUP BY, and to enforce uniqueness.

256. How does an index improve query performance?

It lets the engine jump straight to matching rows (logarithmic lookup) instead of reading every row (full table scan).

257. What are the disadvantages of indexes?

Extra storage, slower INSERT/UPDATE/DELETE because indexes must be maintained, and too many indexes confuse the optimizer and waste resources.

258. What is a clustered index?

An index that defines the physical order of the table's rows; a table can have only one. In MySQL InnoDB the primary key is the clustered index.

259. What is a non-clustered index?

A separate structure holding the indexed columns plus pointers to the rows (in InnoDB, the primary key value). A table can have many.

260. Can a table have multiple indexes?

Yes, many non-clustered/secondary indexes (but only one clustered).

261. Can indexes slow down INSERT/UPDATE operations?

Yes. Every write must also update each affected index, so write-heavy tables should have only necessary indexes.

262. Which columns should be indexed?

Columns frequently used in WHERE, JOIN, ORDER BY and GROUP BY, especially high-cardinality columns (many distinct values) and foreign keys. Avoid indexing tiny or rarely used columns.

263. What is a composite index?

An index on multiple columns, e.g. `(customer_id, order_date)`. It follows the leftmost-prefix rule: it helps queries filtering on customer_id, or customer_id + order_date, but not order_date alone.

264. What is a unique index?

An index that also forbids duplicate values in the indexed column(s). A UNIQUE constraint creates one.

18. Query Performance & Optimization

265. ★ How do you optimize a SQL query?

- Select only needed columns and filter early.
- Index columns used in WHERE/JOIN/ORDER BY.
- Avoid functions on indexed columns in WHERE.
- Prefer JOIN/EXISTS patterns that suit the data; use UNION ALL over UNION when possible.
- Reduce joined data volume with CTEs/pre-aggregation.
- Read the execution plan with EXPLAIN and fix full scans.

266. ★ Why should you avoid `SELECT *`?

It reads and transfers unneeded columns, prevents covering-index use, adds memory/network load and breaks code when the schema changes.

267. What is an execution plan?

The step-by-step strategy the optimizer chooses to run a query: access method, join order, indexes used and estimated rows. It shows where time is spent.

268. What is `EXPLAIN` in MySQL?

A keyword placed before a query (`EXPLAIN SELECT ...`) that shows its execution plan. Look at `type` (ALL = full scan is bad), `key` (index used), `rows` and `Extra`. `EXPLAIN ANALYZE` (8.0.18+) also runs it and shows actual timings.

269. How can indexes improve performance?

By turning full table scans into index seeks for filters, joins and sorts, and sometimes covering the query entirely from the index.

270. What causes a query to run slowly?

Missing or unused indexes, full table scans, `SELECT *`, functions on indexed columns, large unfiltered joins, row-by-row correlated subqueries, no pre-aggregation, huge sorts, outdated statistics or too much data returned.

271. How can you optimize a JOIN?

Index the join columns, join on matching data types, filter before joining, join only needed columns/rows, and check join order and plan with EXPLAIN.

272. How can you optimize a GROUP BY query?

Filter with WHERE first, index the grouping columns, group on as few columns as needed, and avoid grouping on computed expressions where possible.

273. Why can functions on indexed columns hurt performance?

Wrapping a column in a function (e.g. `WHERE YEAR(order_date) = 2025`) stops the optimizer from using the index (the condition is not "sargable"). Rewrite as a range: `order_date >= '2025-01-01' AND order_date < '2026-01-01'`.

274. What is a full table scan?

Reading every row of a table to find matches, which is slow on large tables. It shows as `type = ALL` in EXPLAIN.

275. What is query optimization?

The process of making a query use fewer resources and run faster, both by rewriting the SQL and by tuning schema/indexes. The database optimizer also does automatic cost-based optimization.

276. How can you reduce unnecessary data processing?

Filter early, select needed columns, aggregate before joining, use LIMIT during development, partition large tables, and avoid repeated calculations by using CTEs or temp tables.

277. When should you use a CTE versus a subquery?

Use a CTE for readability, when the result is reused multiple times, or for recursion. Use a simple inline subquery for short one-off filters. Performance is generally similar in MySQL 8.

278. How can you identify performance bottlenecks?

Use EXPLAIN/EXPLAIN ANALYZE, the slow query log, SHOW PROFILE/Performance Schema, and check for full scans, large row estimates, filesorts and temporary tables.

19. Transactions

279. ★ What is a transaction?

A group of SQL statements executed as a single unit of work: either all succeed (commit) or none do (rollback). Example: transferring money debits one account and credits another together.

280. ★ What are ACID properties?

Atomicity, Consistency, Isolation, Durability: the guarantees that make transactions reliable.

281. What is COMMIT?

Permanently saves all changes made in the current transaction.

282. What is ROLLBACK?

Undoes all uncommitted changes made in the current transaction, returning the database to its previous state.

283. What is SAVEPOINT?

A marker inside a transaction you can roll back to without undoing the whole transaction: `SAVEPOINT sp1; ... ROLLBACK TO sp1;`

284. What is transaction isolation?

The degree to which a transaction is shielded from others running concurrently. Levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ (MySQL default), SERIALIZABLE; stricter levels prevent dirty reads, non-repeatable reads and phantom reads.

285. What is atomicity?

All-or-nothing: a transaction either completes fully or has no effect.

286. What is consistency?

A transaction takes the database from one valid state to another, respecting all constraints and rules.

287. What is isolation?

Concurrent transactions do not interfere with each other; each behaves as if it ran alone (to the degree of the isolation level).

288. What is durability?

Once committed, changes persist even after a crash or power loss (via logs written to disk).

20. Frequently Asked SQL Coding Questions

Interviewers often hand you a table and ask you to write the query. All solutions use the sample schema on page 2 (MySQL 8 syntax). Replace N with the value you need.

289. ★ Find the second-highest salary.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Window-function alternative (handles any N):

```
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;
```

290. ★ Find the third-highest salary.

```
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 3;
```

291. ★ Find the Nth-highest salary.

Use DENSE_RANK so duplicate salaries count as one rank. Change `N` to the rank you need.

```
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = N;
```

Without window functions: `SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET N-1;` (replace N-1 with the number).

292. ★ Find the highest salary in each department.

```
SELECT dept_id, MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id;
```

To also show who earns it:

```
SELECT e.*
FROM employees e
JOIN (SELECT dept_id, MAX(salary) AS max_sal FROM employees GROUP BY dept_id) m
  ON e.dept_id = m.dept_id AND e.salary = m.max_sal;
```

293. ★ Find employees earning more than the average salary.

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

294. ★ Find duplicate records.

Group by all columns that define a duplicate.

```
SELECT emp_name, dept_id, salary, COUNT(*) AS cnt
FROM employees
GROUP BY emp_name, dept_id, salary
HAVING COUNT(*) > 1;
```

295. ★ Find employees who don't have a department.

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

296. ★ Find customers who never placed an order.

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

297. ★ Find the top 5 highest-paid employees.

```
SELECT emp_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

If ties at the 5th salary should all be included, use `DENSE_RANK() OVER (ORDER BY salary DESC) <= 5` in a subquery instead.

298. ★ Find the number of employees in each department.

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

299. ★ Find departments with more than 5 employees.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

300. ★ Find the average salary by department.

```
SELECT dept_id, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY dept_id;
```

21. Intermediate SQL Problems

301. ★ Find the second-highest salary in each department.

```
SELECT dept_id, emp_name, salary
FROM (
    SELECT dept_id, emp_name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;
```

302. ★ Find the top 3 salaries in each department.

```
SELECT dept_id, emp_name, salary
FROM (
    SELECT dept_id, emp_name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk <= 3;
```

303. Find employees who earn more than their manager.

Self join employees to their manager.

```
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

304. Find employees who earn the same salary.

```
SELECT emp_name, salary
FROM employees
WHERE salary IN (SELECT salary FROM employees GROUP BY salary HAVING COUNT(*) > 1)
ORDER BY salary;
```

305. Find the department with the highest average salary.

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC
LIMIT 1;
```

Add a RANK()-based version if ties should all be returned.

306. Find the department with the highest number of employees.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
ORDER BY emp_count DESC
LIMIT 1;
```

307. Find customers with more than 3 orders.

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

308. Find customers whose total spending is above average.

```
WITH spend AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders GROUP BY customer_id
)
SELECT customer_id, total_spent
FROM spend
WHERE total_spent > (SELECT AVG(total_spent) FROM spend);
```

309. Find the most expensive product in each category.

```
SELECT category, product_name, price
FROM (
    SELECT category, product_name, price,
           RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rnk
    FROM products
) t
WHERE rnk = 1;
```

310. Find the second-most expensive product in each category.

```
SELECT category, product_name, price
FROM (
    SELECT category, product_name, price,
           DENSE_RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rnk
    FROM products
) t
WHERE rnk = 2;
```

311. Find the highest-selling product.

Here "highest-selling" is measured by units sold; swap in revenue if you prefer.

```
SELECT p.product_name, SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY units_sold DESC
LIMIT 1;
```

312. Find the top 3 products by sales.

```
SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY revenue DESC
LIMIT 3;
```

313. Find products that have never been sold.

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

314. Find customers who ordered in January but not February.

Example for 2025. NOT EXISTS is safer than NOT IN.

```
SELECT DISTINCT o1.customer_id
FROM orders o1
WHERE o1.order_date >= '2025-01-01' AND o1.order_date < '2025-02-01'
  AND NOT EXISTS (
      SELECT 1 FROM orders o2
      WHERE o2.customer_id = o1.customer_id
        AND o2.order_date >= '2025-02-01' AND o2.order_date < '2025-03-01'
  );
```

315. Find customers who ordered every month.

Compare each customer's distinct months with the total months in the data.

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_FORMAT(order_date, '%Y-%m')) =
       (SELECT COUNT(DISTINCT DATE_FORMAT(order_date, '%Y-%m')) FROM orders);
```

22. Advanced Analytical SQL Questions

316. ★ Calculate month-over-month sales growth.

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS sales
    FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT ym, sales,
       ROUND(100 * (sales - LAG(sales) OVER (ORDER BY ym))
             / NULLIF(LAG(sales) OVER (ORDER BY ym), 0), 2) AS mom_growth_pct
FROM monthly;
```

317. ★ Calculate year-over-year sales growth.

```
WITH yearly AS (
    SELECT YEAR(order_date) AS yr, SUM(amount) AS sales
    FROM orders GROUP BY YEAR(order_date)
)
SELECT yr, sales,
       ROUND(100 * (sales - LAG(sales) OVER (ORDER BY yr))
             / NULLIF(LAG(sales) OVER (ORDER BY yr), 0), 2) AS yoy_growth_pct
FROM yearly;
```

For month vs same month last year, use `LAG(sales, 12)` on a monthly series (only valid when no month is missing), or self join on `ym = DATE_ADD(prev, INTERVAL 1 YEAR)`.

318. ★ Calculate a running total.

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date, order_id) AS running_total
FROM orders;
```

319. ★ Calculate a 7-day moving average.

Aggregate to daily totals first, then use a date-range frame.

```
WITH daily AS (
    SELECT order_date, SUM(amount) AS daily_sales
    FROM orders GROUP BY order_date
)
SELECT order_date, daily_sales,
       ROUND(AVG(daily_sales) OVER (
           ORDER BY order_date
           RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW), 2) AS moving_avg_7d
FROM daily;
```

320. ★ Find the top 3 products in every category.

```
WITH prod_sales AS (
    SELECT p.category, p.product_name,
           SUM(oi.quantity * oi.unit_price) AS revenue
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category, p.product_name
),
ranked AS (
    SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rnk
    FROM prod_sales
)
SELECT category, product_name, revenue
FROM ranked
WHERE rnk <= 3;
```

321. ★ Find the first order of every customer.

```
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date, order_id) AS rn
    FROM orders o
) t WHERE rn = 1;
```

322. ★ Find the latest order of every customer.

```
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date DESC, order_id DESC) AS rn
    FROM orders o
) t WHERE rn = 1;
```

323. ★ Find customers' previous order date.

```
SELECT customer_id, order_id, order_date,
       LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date, order_id) AS prev_order_date
FROM orders;
```

324. ★ Calculate the number of days between customer orders.

```
SELECT customer_id, order_id, order_date,
       DATEDIFF(order_date,
                LAG(order_date) OVER (PARTITION BY customer_id
                                      ORDER BY order_date, order_id)) AS days_since_prev
FROM orders;
```

325. ★ Find customers who made purchases in consecutive months.

```
WITH m AS (
    SELECT DISTINCT customer_id,
           DATE_SUB(order_date, INTERVAL DAY(order_date) - 1 DAY) AS mth
    FROM orders
)
SELECT DISTINCT a.customer_id
FROM m a
JOIN m b ON a.customer_id = b.customer_id
        AND b.mth = DATE_ADD(a.mth, INTERVAL 1 MONTH);
```

326. Find the longest consecutive purchasing streak.

Use the "gaps and islands" technique: convert months to an index; consecutive months have the same (index minus row number).

```
WITH m AS (
    SELECT DISTINCT customer_id,
           YEAR(order_date) * 12 + MONTH(order_date) AS mi
    FROM orders
),
g AS (
    SELECT customer_id, mi,
           mi - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY mi) AS grp
    FROM m
),
streaks AS (
    SELECT customer_id, grp, COUNT(*) AS streak_len
    FROM g GROUP BY customer_id, grp
)
SELECT customer_id, MAX(streak_len) AS longest_streak_months
FROM streaks
GROUP BY customer_id
ORDER BY longest_streak_months DESC;
```

Add `LIMIT 1` for the single longest streak overall.

327. Find customers whose spending increased every month.

```
WITH ms AS (
    SELECT customer_id, DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS spend
    FROM orders GROUP BY customer_id, DATE_FORMAT(order_date, '%Y-%m')
),
chg AS (
    SELECT customer_id, ym, spend,
           LAG(spend) OVER (PARTITION BY customer_id ORDER BY ym) AS prev_spend
    FROM ms
)
SELECT customer_id
FROM chg
GROUP BY customer_id
HAVING COUNT(prev_spend) >= 1
   AND SUM(CASE WHEN spend <= prev_spend THEN 1 ELSE 0 END) = 0;
```

328. Find the percentage contribution of each product to total sales.

```
SELECT p.product_name,
       SUM(oi.quantity * oi.unit_price) AS revenue,
       ROUND(100 * SUM(oi.quantity * oi.unit_price)
             / SUM(SUM(oi.quantity * oi.unit_price)) OVER (), 2) AS pct_of_total
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY pct_of_total DESC;
```

329. Find each department's percentage contribution to company salary.

```
SELECT dept_id, SUM(salary) AS dept_salary,
       ROUND(100 * SUM(salary) / SUM(SUM(salary)) OVER (), 2) AS pct_of_company
FROM employees
GROUP BY dept_id;
```

330. Find the cumulative percentage of sales.

Rank products by revenue and accumulate (Pareto / ABC analysis).

```
WITH rev AS (
    SELECT product_id, SUM(quantity * unit_price) AS revenue
    FROM order_items GROUP BY product_id
)
SELECT product_id, revenue,
       ROUND(100 * SUM(revenue) OVER (ORDER BY revenue DESC, product_id)
             / SUM(revenue) OVER (), 2) AS cumulative_pct
FROM rev
ORDER BY revenue DESC, product_id;
```

331. Find the median salary.

```
WITH r AS (
    SELECT salary,
           ROW_NUMBER() OVER (ORDER BY salary) AS rn,
           COUNT(*) OVER () AS cnt
    FROM employees
)
SELECT AVG(salary) AS median_salary
FROM r
WHERE rn IN (FLOOR((cnt + 1) / 2), CEIL((cnt + 1) / 2));
```

332. Find the mode of a column.

```
SELECT salary, COUNT(*) AS freq
FROM employees
GROUP BY salary
HAVING COUNT(*) = (
    SELECT MAX(c) FROM (SELECT COUNT(*) AS c FROM employees GROUP BY salary) x
);
```

The HAVING clause returns all tied modes; use ORDER BY freq DESC LIMIT 1 for just one.

333. Find the most frequently purchased product.

```
SELECT p.product_name, COUNT(*) AS times_purchased
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY times_purchased DESC
LIMIT 1;
```

334. Find the customer with the highest lifetime value.

```
SELECT customer_id, SUM(amount) AS lifetime_value
FROM orders
GROUP BY customer_id
ORDER BY lifetime_value DESC
LIMIT 1;
```

335. Find customers whose latest order value is greater than their average order value.

```
WITH x AS (
    SELECT customer_id, amount,
           ROW_NUMBER() OVER (PARTITION BY customer_id
                              ORDER BY order_date DESC, order_id DESC) AS rn,
           AVG(amount) OVER (PARTITION BY customer_id) AS avg_amount
    FROM orders
)
SELECT customer_id, amount AS latest_amount, ROUND(avg_amount, 2) AS avg_amount
FROM x
WHERE rn = 1 AND amount > avg_amount;
```

23. Business/Data Analyst SQL Questions

These test whether you can use SQL for analysis. State your metric definition before writing the query. Where the schema lacks a needed table, the assumption is noted.

336. How would you calculate monthly revenue?

Sum order amounts per month, usually excluding cancelled orders.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(amount) AS revenue
FROM orders
WHERE status <> 'Cancelled'
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

337. How would you calculate customer retention?

Retention = share of a cohort (customers by first-order month) that is active again in later months. This gives active customers per cohort and months since first order; divide by the month-0 count for a percentage.

```
WITH first_m AS (
    SELECT customer_id,
           DATE_SUB(MIN(order_date), INTERVAL DAY(MIN(order_date)) - 1 DAY) AS cohort
    FROM orders GROUP BY customer_id
),
act AS (
    SELECT DISTINCT customer_id,
           DATE_SUB(order_date, INTERVAL DAY(order_date) - 1 DAY) AS mth
    FROM orders
)
SELECT f.cohort,
       TIMESTAMPDIFF(MONTH, f.cohort, a.mth) AS months_since_first,
       COUNT(DISTINCT a.customer_id) AS active_customers
FROM first_m f
JOIN act a ON f.customer_id = a.customer_id
GROUP BY f.cohort, TIMESTAMPDIFF(MONTH, f.cohort, a.mth)
ORDER BY f.cohort, months_since_first;
```

338. How would you calculate customer churn?

Churn rate = customers active last period who are not active this period, divided by customers active last period. Example (Jan 2025 to Feb 2025):

```
SELECT ROUND(100 * SUM(NOT EXISTS (
           SELECT 1 FROM orders f
           WHERE f.customer_id = j.customer_id
             AND f.order_date >= '2025-02-01' AND f.order_date < '2025-03-01'
       )) / COUNT(*), 2) AS churn_pct
FROM (SELECT DISTINCT customer_id FROM orders
      WHERE order_date >= '2025-01-01' AND order_date < '2025-02-01') j;
```

Many businesses instead define churn as "no purchase in the last 90 days" (see inactive customers below).

339. How would you calculate Customer Lifetime Value?

Historical CLV is total revenue per customer. Predictive CLV is roughly average order value x purchase frequency x expected customer lifespan (optionally x margin).

```
SELECT customer_id,
       COUNT(*)            AS orders_cnt,
       SUM(amount)         AS lifetime_value,
       ROUND(AVG(amount), 2) AS avg_order_value
FROM orders
GROUP BY customer_id
ORDER BY lifetime_value DESC;
```

340. How would you identify your top customers?

Rank by total spend (and/or order count) and take the top N or top 10%.

```
SELECT customer_id, SUM(amount) AS total_spent,
       DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS spend_rank
FROM orders
GROUP BY customer_id
ORDER BY spend_rank
LIMIT 10;
```

341. How would you identify inactive customers?

Define inactivity (e.g. no order in 90 days), then find customers whose latest order is older than that, including those who never ordered.

```
SELECT c.customer_id, c.customer_name, MAX(o.order_date) AS last_order
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING MAX(o.order_date) IS NULL
    OR MAX(o.order_date) < CURDATE() - INTERVAL 90 DAY;
```

342. How would you identify repeat customers?

Customers with more than one order.

```sql
SELECT customer_id, COUNT(*) AS orders_cnt
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

343. How would you calculate average order value?

AOV = total revenue / number of orders.

```sql
SELECT ROUND(SUM(amount) / COUNT(DISTINCT order_id), 2) AS aov
FROM orders;
```

344. How would you calculate conversion rate?

Conversion rate = visitors who completed the goal (purchase) / total visitors. Assumes a `visits(user_id, visit_date)` table where user_id matches customer_id.

```sql
SELECT ROUND(100 * COUNT(DISTINCT o.customer_id) / COUNT(DISTINCT v.user_id), 2) AS conversion_pct
FROM visits v
LEFT JOIN orders o ON o.customer_id = v.user_id;
```

345. How would you calculate cancellation rate?

Cancelled orders divided by total orders.

```sql
SELECT ROUND(100 * SUM(status = 'Cancelled') / COUNT(*), 2) AS cancellation_rate_pct
FROM orders;
```

346. How would you calculate product return rate?

Returned units / sold units per product. Assumes a `returns(order_id, product_id, quantity)` table.

```sql
SELECT p.product_name,
       ROUND(100 * COALESCE(SUM(r.quantity), 0) / SUM(oi.quantity), 2) AS return_rate_pct
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN returns r ON r.order_id = oi.order_id AND r.product_id = oi.product_id
GROUP BY p.product_id, p.product_name;
```

347. How would you identify the best-performing product?

Rank products by revenue (or units or profit) and take the top.

```sql
SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY revenue DESC
LIMIT 1;
```

348. How would you identify the worst-performing product?

Same query ordered ascending, but include products with zero sales via a LEFT JOIN.

```sql
SELECT p.product_name, COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY revenue ASC
LIMIT 1;
```

349. How would you identify the best-performing region?

```sql
SELECT c.region, SUM(o.amount) AS revenue
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.region
ORDER BY revenue DESC
LIMIT 1;
```

350. How would you identify declining sales?

Compare each month with the previous month and flag decreases.

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS sales
    FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT ym, sales, prev_sales
FROM (SELECT ym, sales, LAG(sales) OVER (ORDER BY ym) AS prev_sales FROM monthly) t
WHERE sales < prev_sales;
```

351. How would you compare current month sales with previous month?

```
SELECT SUM(CASE WHEN order_date >= DATE_FORMAT(CURDATE(), '%Y-%m-01')
                THEN amount END) AS this_month,
       SUM(CASE WHEN order_date <  DATE_FORMAT(CURDATE(), '%Y-%m-01')
                THEN amount END) AS prev_month
FROM orders
WHERE order_date >= DATE_FORMAT(CURDATE(), '%Y-%m-01') - INTERVAL 1 MONTH
  AND order_date <  DATE_FORMAT(CURDATE(), '%Y-%m-01') + INTERVAL 1 MONTH;
```

352. How would you identify customers whose spending has decreased?

Compare each customer's monthly spend with their previous month's spend.

```
WITH ms AS (
    SELECT customer_id, DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS spend
    FROM orders GROUP BY customer_id, DATE_FORMAT(order_date, '%Y-%m')
)
SELECT customer_id, ym, spend, prev_spend
FROM (SELECT customer_id, ym, spend,
             LAG(spend) OVER (PARTITION BY customer_id ORDER BY ym) AS prev_spend
      FROM ms) t
WHERE spend < prev_spend;
```

353. How would you calculate the percentage of total sales by category?

```
SELECT p.category,
       SUM(oi.quantity * oi.unit_price) AS revenue,
       ROUND(100 * SUM(oi.quantity * oi.unit_price)
             / SUM(SUM(oi.quantity * oi.unit_price)) OVER (), 2) AS pct_of_total
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category;
```

354. How would you find the most profitable category?

Profit = revenue minus cost. Assumes a `cost` column on products.

```
SELECT p.category,
       SUM(oi.quantity * (oi.unit_price - p.cost)) AS profit
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY profit DESC
LIMIT 1;
```

355. How would you find the most active customers?

Define "active" (order count, distinct active months, or logins) and rank by it.

```
SELECT customer_id,
       COUNT(*) AS orders_cnt,
       COUNT(DISTINCT DATE_FORMAT(order_date, '%Y-%m')) AS active_months
FROM orders
GROUP BY customer_id
ORDER BY orders_cnt DESC, active_months DESC
LIMIT 10;
```

24. SQL + Real-World Scenarios

356. A company has customers and orders tables. How would you find customers who haven't purchased anything?

Anti-join with LEFT JOIN ... IS NULL (or NOT EXISTS).

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

357. You have an employee table. How would you find employees who earn more than their manager?

Self join the table on manager_id and compare salaries.

```
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

358. You have sales data. How would you find the top 3 products every month?

Aggregate by month and product, then rank within each month.

```
WITH ps AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS ym, oi.product_id,
           SUM(oi.quantity * oi.unit_price) AS revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m'), oi.product_id
)
SELECT ym, product_id, revenue
FROM (SELECT *, DENSE_RANK() OVER (PARTITION BY ym ORDER BY revenue DESC) AS rnk FROM ps) t
WHERE rnk <= 3
ORDER BY ym, rnk;
```

359. You have website traffic data. How would you calculate daily active users?

Count distinct users per day. Assumes `logins(user_id, login_date)`.

```
SELECT login_date, COUNT(DISTINCT user_id) AS dau
FROM logins
GROUP BY login_date
ORDER BY login_date;
```

360. How would you find users who logged in for 3 consecutive days?

Gaps-and-islands: consecutive dates produce the same (date minus row number) value.

```
WITH d AS (
    SELECT user_id, login_date,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
    FROM (SELECT DISTINCT user_id, login_date FROM logins) x
),
g AS (
    SELECT user_id, login_date, DATE_SUB(login_date, INTERVAL rn DAY) AS grp
    FROM d
)
SELECT DISTINCT user_id
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

361. How would you calculate monthly active users?

```
SELECT DATE_FORMAT(login_date, '%Y-%m') AS month, COUNT(DISTINCT user_id) AS mau
FROM logins
GROUP BY DATE_FORMAT(login_date, '%Y-%m')
ORDER BY month;
```

362. How would you identify users who became inactive?

Users whose last login is older than your inactivity threshold (e.g. 30 days).

```
SELECT user_id, MAX(login_date) AS last_login
FROM logins
GROUP BY user_id
HAVING MAX(login_date) < CURDATE() - INTERVAL 30 DAY;
```

363. How would you calculate customer retention after 30 days?

Share of users who return on day 30 after signup. Assumes `users(user_id, signup_date)`; widen the join to a day 30-37 window for a more forgiving definition.

```
SELECT ROUND(100 * COUNT(DISTINCT l.user_id) / COUNT(DISTINCT u.user_id), 2) AS d30_retention_pct
FROM users u
LEFT JOIN logins l
       ON l.user_id = u.user_id
      AND l.login_date = DATE_ADD(u.signup_date, INTERVAL 30 DAY);
```

364. How would you calculate average order value?

`SELECT ROUND(SUM(amount) / COUNT(DISTINCT order_id), 2) AS aov FROM orders;` (or AVG(amount) when each row is one order).

365. How would you calculate revenue growth?

Aggregate revenue per period, then compare with the previous period using LAG.

```
WITH monthly AS (
    SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS revenue
    FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT ym, revenue,
       ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY ym))
             / NULLIF(LAG(revenue) OVER (ORDER BY ym), 0), 2) AS growth_pct
FROM monthly;
```

366. How would you find the first purchase made by each customer?

```
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date, order_id) AS rn
    FROM orders o
) t WHERE rn = 1;
```

367. How would you identify repeat purchases?

Number each customer's orders; any order numbered 2 or higher is a repeat purchase.

```
SELECT * FROM (
    SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date, order_id) AS order_no
    FROM orders o
) t WHERE order_no > 1;
```

368. How would you identify duplicate transactions?

Group by the fields that should be unique together (customer, amount, timestamp) and find groups with more than one row.

```
SELECT customer_id, order_date, amount, COUNT(*) AS cnt
FROM orders
GROUP BY customer_id, order_date, amount
HAVING COUNT(*) > 1;
```

369. How would you identify transactions that occurred more than once?

Same idea on the transaction identifier, then list the full rows.

```
SELECT o.*
FROM orders o
JOIN (SELECT order_id FROM orders GROUP BY order_id HAVING COUNT(*) > 1) d
  ON o.order_id = d.order_id;
```

370. How would you find the most popular product by month?

```
WITH pm AS (
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS ym, oi.product_id,
           SUM(oi.quantity) AS units
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m'), oi.product_id
)
SELECT ym, product_id, units
FROM (SELECT *, RANK() OVER (PARTITION BY ym ORDER BY units DESC) AS rnk FROM pm) t
WHERE rnk = 1;
```

371. How would you find the highest revenue-generating customer in each region?

```
WITH cr AS (
    SELECT c.region, c.customer_id, c.customer_name, SUM(o.amount) AS revenue
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.region, c.customer_id, c.customer_name
)
SELECT region, customer_id, customer_name, revenue
FROM (SELECT *, RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS rnk FROM cr) t
WHERE rnk = 1;
```

25. MySQL-Specific Questions

372. ★ What is MySQL?

An open-source relational database management system (owned by Oracle) that stores data in tables and is queried with SQL. Its default storage engine is InnoDB, which supports transactions, foreign keys and row-level locking.

373. What are MySQL data types?

- Numeric: TINYINT, INT, BIGINT, DECIMAL, FLOAT, DOUBLE.
- String: CHAR, VARCHAR, TEXT, ENUM, SET.
- Date/time: DATE, TIME, DATETIME, TIMESTAMP, YEAR.
- Other: JSON, BLOB, BOOLEAN (alias for TINYINT(1)).

374. Difference between `CHAR` and `VARCHAR`?

CHAR is fixed length (padded with spaces), best for constant-length values like country codes. VARCHAR is variable length and uses only the space needed, best for names and emails.

375. Difference between `DATE`, `DATETIME`, and `TIMESTAMP`?

- DATE: only the date (YYYY-MM-DD).
- DATETIME: date and time, no time-zone conversion, range 1000 to 9999.
- TIMESTAMP: date and time stored as UTC and converted to the session time zone; range 1970 to 2038.

376. What is `AUTO_INCREMENT`?

A column attribute that automatically generates a unique, increasing number for each new row, typically for surrogate primary keys: `id INT AUTO_INCREMENT PRIMARY KEY`.

377. What is `LIMIT`?

Restricts how many rows a query returns: `LIMIT 10` or `LIMIT 10 OFFSET 20`. Always use with ORDER BY for predictable results.

378. What is `IFNULL()`?

`IFNULL(expr, alt)` returns `alt` when `expr` is NULL, otherwise `expr`.

379. What is `COALESCE()`?

Returns the first non-NULL value from a list of arguments; standard SQL and more flexible than IFNULL.

380. What is `STR_TO_DATE()`?

Converts a text value to a date using a format string: `STR_TO_DATE('15/08/2025', '%d/%m/%Y')` returns 2025-08-15.

381. What is `DATE_FORMAT()`?

Formats a date as text: `DATE_FORMAT(order_date, '%Y-%m')` returns 2025-08; `'%d %b %Y'` returns 15 Aug 2025. Common codes: %Y year, %m month, %d day, %M month name, %W weekday name.

382. What is `TIMESTAMPDIFF()`?

Returns the difference between two datetimes in the unit you choose: `TIMESTAMPDIFF(DAY, start_date, end_date)`.

383. What is `GROUP_CONCAT()`?

An aggregate that joins values from a group into one string: `SELECT dept_id, GROUP_CONCAT(emp_name ORDER BY emp_name SEPARATOR ', ') FROM employees GROUP BY dept_id;` Output length is limited by `group_concat_max_len`.

384. What is `SUBSTRING_INDEX()`?

Returns part of a string before or after the Nth delimiter: `SUBSTRING_INDEX('a@b.com', '@', 1)` gives a; with -1 gives b.com.

385. What is `REGEXP`?

Regular-expression pattern matching: `WHERE email REGEXP '^[a-z0-9._]+@[a-z]+\\.com$'`. `REGEXP_LIKE()`, `REGEXP_REPLACE()` and `REGEXP_SUBSTR()` are available in MySQL 8.

386. What is `EXPLAIN`?

Shows the execution plan for a query: which indexes are used, join order and estimated rows. Used to diagnose slow queries.

387. What is `LOAD DATA INFILE`?

A fast bulk-import command that loads rows from a text/CSV file into a table.

```
LOAD DATA INFILE '/path/data.csv'
INTO TABLE customers
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Use `LOAD DATA LOCAL INFILE` for a file on the client machine (it must be enabled), and check `secure_file_priv` for server-side paths.

388. What is `ONLY_FULL_GROUP_BY`?

A default SQL mode in MySQL 5.7+ that rejects a query if a selected non-aggregated column is not in GROUP BY (or functionally dependent on it). It prevents ambiguous results; fix the query rather than disabling the mode.

389. What is the difference between `VARCHAR` and `TEXT`?

VARCHAR(n) has a declared maximum length, can have a DEFAULT value and can be fully indexed. TEXT is for long text (up to 64 KB), cannot have a default value in older versions and needs a prefix length to be indexed.

390. What are stored procedures?

Saved, named blocks of SQL that run on the server and can take parameters, hold logic (IF, loops) and run several statements. Called with `CALL proc_name(args);`. They enable reuse, security and fewer client round trips.

391. What are triggers?

Code that runs automatically BEFORE or AFTER an INSERT, UPDATE or DELETE on a table, used for auditing, validation and keeping derived data in sync.

392. What are functions in MySQL?

Built-in functions (e.g. SUM, CONCAT, DATE_FORMAT) or user-defined functions (`CREATE FUNCTION`) that return a single value and can be used inside queries, unlike stored procedures, which are called separately with CALL.

Bonus: Extra Question from the "First 25" List

Bonus. ★ DELETE vs DROP vs TRUNCATE?

- DELETE (DML): removes selected rows using WHERE; can be rolled back; fires triggers; keeps table structure; AUTO_INCREMENT counter is not reset; slower on large tables.
- TRUNCATE (DDL): removes all rows quickly; no WHERE clause; resets AUTO_INCREMENT; in MySQL it cannot be rolled back (implicit commit); keeps the table structure.
- DROP (DDL): deletes the entire table (data, structure, indexes, privileges); cannot be rolled back.

Quick Checklist: What to Master First

Must know

- SELECT, WHERE, DISTINCT, ORDER BY, LIMIT
- GROUP BY, HAVING, COUNT / SUM / AVG / MIN / MAX
- CASE and NULL handling (COALESCE, IFNULL)
- INNER, LEFT and RIGHT JOIN; anti-joins; self joins
- Subqueries, EXISTS vs IN, CTEs
- UNION vs UNION ALL
- ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, running totals
- Date and string functions; duplicate detection and data cleaning
- Second-highest salary, Top N, Top N per group, month-over-month analysis

Good to know

- Recursive CTEs, views, indexes and EXPLAIN
- Transactions and ACID
- Stored procedures and triggers
- Advanced window frames (moving averages, FIRST_VALUE / LAST_VALUE)